Real-time Vocal Autotuning and Vocoding

The VoCoder's: Kalyan Sriram, Avery Dudra, Vishrut Agrawal, Jonathan Wang
ECE 554: Digital Engineering Laboratory
Github Link: GitHub

1. Objective

Digital audio processing has historically been constrained to offline or high-latency software environments, limiting its application in live performance contexts. This project aims to bring vocal effects such as pitch correction (autotune) and channel vocoding to a real-time hardware implementation on an FPGA platform, enabling low-latency audio processing suitable for live use.

The core technical goals of this project are as follows. First, to implement a real-time pitch detection and correction pipeline capable of identifying the fundamental frequency of an incoming vocal signal and shifting it to the nearest target pitch with minimal latency. Second, to implement a channel vocoder that extracts the spectral envelope of a modulator signal and applies it to a carrier signal, producing the characteristic vocoded output. Finally, to verify the correctness of each processing stage through simulation before validating the complete system on hardware.

Together these goals target a complete, integrated real-time vocal processing system implemented in synthesizable RTL, with a design methodology that prioritizes fixed-point numerical correctness, hardware resource efficiency, and stable audio output across all operating modes.

2. Background

The system is built on three core digital signal processing techniques. Autocorrelation forms the basis of the pitch detection stage, providing a time-domain method for estimating the fundamental frequency of an incoming vocal signal on a frame-by-frame basis. TD-PSOLA builds on this pitch estimate to perform pitch correction, manipulating the temporal structure of the signal to shift its perceived pitch toward a target musical note while preserving its natural timbral character. Finally, channel vocoding provides the mechanism for the vocoder effect, using the spectral envelope of one signal to modulate the amplitude of another across a bank of frequency channels, producing the characteristic effect of a carrier signal taking on the spoken or sung qualities of a vocal input. Together these three techniques form a complete signal processing chain that enables the real-time vocal effects implemented in this project.

2a. Autocorrelation

Autocorrelation is a mathematical technique that measures the similarity of a signal with a delayed version of itself. For a discrete signal x[n], the autocorrelation at lag τ is defined as:

$$r_{xx}[\tau] = \sum_{K} x[k] \cdot x[k+\tau]$$

Where r[τ] is the autocorrelation of the signal at lag τ, x[k] is the input signal at sample index k, x[k+τ] is the input signal shifted by lag τ, and K is the set of sample indices over the analysis window.

The key insight for pitch detection is that a periodic signal will produce a large autocorrelation value when the lag τ corresponds to the fundamental period of the signal. This is because the signal aligns closely with itself when shifted by exactly one period. The autocorrelation function will therefore exhibit

a peak at $\tau = T$, where T is the fundamental period, and the fundamental frequency of the signal. In practice the autocorrelation is computed over a short window of samples (a frame), and the lag of the largest peak after $\tau = 0$ is taken as the estimated period. Care must be taken to ignore the trivial peak at $\tau = 0$, which is always the largest by definition.

Autocorrelation is well suited to hardware implementation because it reduces to a series of multiply-accumulate operations over a sliding window, mapping naturally onto DSP blocks available in FPGA devices. Its primary limitation is a sensitivity to noise and harmonics, which can cause the detector to lock onto a harmonic rather than the true fundamental; this phenomenon is known as octave error.

2b. Time Domain PSOLA

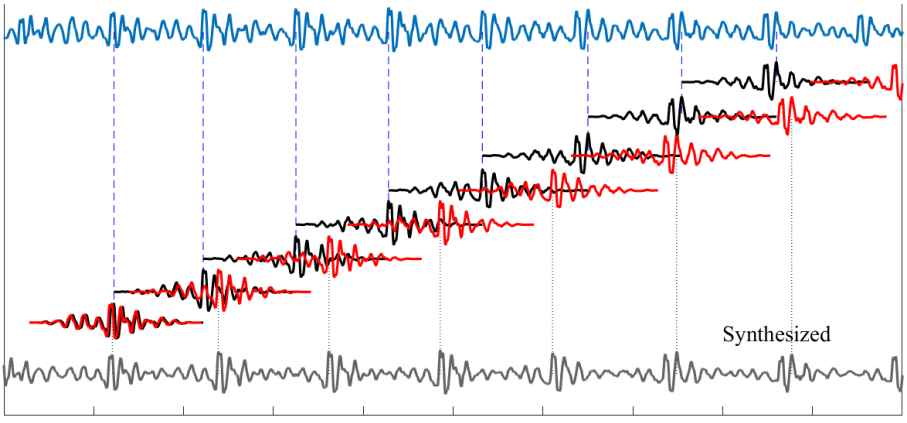
Time Domain Pitch Synchronous Overlap and Add (TD-PSOLA) is a technique for modifying the pitch of a speech or vocal signal without altering its duration or spectral envelope. It operates directly on the time domain waveform, making it computationally efficient relative to frequency domain alternatives.

The algorithm proceeds in three stages. First, the input signal is segmented into a series of short overlapping frames, each centred on a pitch epoch. A pitch epoch is a specific point in the waveform corresponding to a glottal closure event in voiced speech (TODO: change this sentence to read better).. These epochs are identified using the pitch period T estimated by the pitch detector. Each frame has a length of approximately 2T, and successive frames are spaced T apart, ensuring that adjacent frames overlap by 50%.

Second, to shift the pitch upward by a factor of α, the output frames are repositioned closer together, spaced at T/α rather than T. To shift pitch downward, frames are spaced further apart at $T \cdot \alpha$. The frames themselves are not stretched or compressed — only their placement in the output changes.

Third, the repositioned frames are summed together with a windowing function (typically Hann) applied to each frame before addition. Because adjacent frames overlap significantly, the windowing ensures a smooth reconstruction with no discontinuities at frame boundaries. The result is a signal with the same spectral character and duration as the input but with a modified fundamental frequency.

In the context of autotune, the pitch correction factor α is determined by comparing the detected pitch f0 to the nearest note in a target musical scale, and $\alpha = f\_target / f0$. The system continuously updates α each frame as the input pitch changes, producing smooth real-time pitch correction. An image illustrating this algorithm is shown below.



2c. Channel Vocoding

A channel vocoder is a system that transfers the spectral envelope of one signal called the modulator, onto another signal called the carrier, producing an output that combines the timbral character of the modulator with the harmonic content of the carrier. The technique became widely used in music production for its characteristic robotic sound.

The implementation consists of two parallel filter banks, each containing N bandpass filters that together span the audible frequency range. Both the modulator and carrier signals are passed through their respective filter banks simultaneously, producing N frequency band signals for each input. The filters are typically spaced logarithmically to approximate the frequency resolution of human hearing.

For each band i, an envelope follower tracks the instantaneous amplitude of the modulator signal in that band, producing a slowly varying control signal that represents an envelope of how much energy the modulator contains in that frequency region at any given moment. This envelope is then used to scale the corresponding carrier band signal via a multiplier.

The N scaled carrier bands are then summed to produce the final output. The result is a signal whose spectral shape follows the time-varying formant structure of the modulator, which is an input voice, but whose fundamental pitch and harmonic richness come from the carrier, which is a synthesizer.

3. Technical Description

3a. Pitch Detection

The pitch detection pipeline is built around an autocorrelation-based fundamental frequency estimator operating on a sliding window of 1024 samples. Rather than maintaining a single 1024 sample buffer, the circular buffer is partitioned into five 256-sample chunks. Each time a new chunk of 256 samples has been collected, the window advances by one chunk, so the 1024 sample analysis window slides forward in steps of 256 samples at a time. This approach reduces the memory management complexity of a single sample advancing circular buffer while still providing significant frame overlap.

At each window, autocorrelation is computed across a set of candidate lags corresponding to the range of fundamental frequencies of interest. Rather than computing each lag sequentially, the pipeline instantiates 32 independent autocorrelation engines operating in parallel, each responsible for a distinct subset of the candidate lag range. This parallelisation was a deliberate architectural decision driven by the latency requirements of the pitch detection stage. The outputs of the 32 autocorrelation engines are then passed into a peak detection module, which identifies the lag corresponding to the largest autocorrelation value across all engines, excluding the trivial peak at lag zero. This lag is then passed downstream to the pitch correction stage.

3b. Pitch Shifting

The PSOLA module implements a streaming pitch correction processor using a time-domain overlap-add architecture built around a circular sample history buffer and a system of virtual channels. Incoming samples are continuously written into the history buffer, which serves as the source material for grain synthesis. Grains, which are the short overlapping segments of the input signal, are scheduled at intervals determined by the target output pitch period rather than the input pitch period, which is the mechanism by which pitch shifting is achieved. The spacing between successive grains is controlled by

the pitch shift ratio, such that grains spaced more closely together produce a higher output pitch and grains spaced further apart produce a lower one. Each grain is centred on a pitch epoch in the history buffer, with fractional interpolation used to position the epoch at sub-sample accuracy, improving pitch correctness at higher shift ratios.

To manage the overlap of simultaneously active grains in a streaming context, the module maintains a bank of 16 virtual channels, each independently tracking the state of one active grain. When a new grain is scheduled, it is assigned to the next available channel, which is initialised with a read pointer into the history buffer and a Hanning window pointer. Each channel advances its window pointer by one sample per clock cycle and is automatically retired when its window is complete. On every output sample, all active channels contribute their windowed history sample to a common accumulator, producing the final overlap-add sum. The raised cosine shape of the Hanning window ensures that the contributions of overlapping grains sum smoothly at every sample point, eliminating amplitude discontinuities at grain boundaries and producing a clean reconstructed output.

3c. Vocoding

The vocoder module implements a 32-band channel vocoder with an integrated carrier synthesizer, processing audio on a per-sample block basis. Rather than accepting an external carrier signal, the module synthesizes its own carrier internally using a ROM-based polyphonic engine. Pre-computed single-cycle waveforms for each supported MIDI note are stored in a carrier ROM, and on each incoming sample the module iterates across all active MIDI notes, accumulating their individual ROM samples into a composite carrier signal. Each note maintains an independent read pointer that advances and wraps at the note's period boundary, ensuring continuous phase-coherent playback across samples. The result is a polyphonic synthesizer output representing the sum of all currently held notes, which serves as the carrier input to the vocoding stage.

The spectral envelope transfer is performed using a shared 64-bank bandpass filter bank, where 32 banks are allocated to the voice signal and 32 to the carrier signal, both sharing a common set of coefficient ROMs to avoid duplicating filter storage. The voice and carrier signals are processed sequentially through the same physical filter bank hardware, with independent state histories maintained for each path. The voice path uses asymmetric envelope following to better capture the dynamic character of speech, while the carrier path uses symmetric following. Once both signals have been filtered, the module iterates across all 32 bands, using the amplitude envelope to scale the corresponding carrier band. The scaled contributions of all 32 bands are accumulated to produce the final vocoded output sample.

4. Evaluation Highlights

A key component of the verification strategy for this project was the development of streaming-based Python models for each of the three processing stages prior to any RTL implementation. Rather than treating each algorithm as a block processor operating on a complete audio buffer, the Python models were written to process one sample at a time, mirroring the streaming architecture that would ultimately be implemented in hardware. This allowed algorithmic correctness to be established in a high level environment where debugging is significantly faster, and ensured that the streaming adaptations made to inherently block-based algorithms such as TD-PSOLA were validated before committing to an RTL implementation.

The output of these Python models served as a golden reference throughout the RTL verification process. Simulation testbenches were driven with the same input stimuli used in the Python models, and the RTL outputs were compared against the Python reference to identify any numerical discrepancies introduced by fixed-point quantisation or pipeline timing. This methodology allowed bugs to be localised to specific stages of the processing chain with confidence, since any deviation from the reference could be attributed to the hardware implementation rather than the underlying algorithm.

5. Challenges, Workarounds, and Lessons Learned

One of the most significant challenges encountered during this project was the lack of detailed implementation documentation for TD-PSOLA. While the algorithm is well established in the literature, the available resources largely provide a high-level conceptual overview of the technique. Most literature describes the epoch-based segmentation and overlap-add reconstruction in general terms focusing on a block-based offline processing model, where the entire audio signal is available prior to analysis. Adapting an inherently frame-based algorithm to operate on a continuous sample stream required considerable design effort beyond what the literature directly supported. A substantial portion of the project timeline was therefore spent whiteboarding the streaming architecture before any RTL was written. This experience highlighted the importance of thoroughly resolving architectural questions at the design stage rather than during implementation, as fundamental structural decisions in a hardware design are costly to revisit once RTL development is underway.

A second significant challenge was managing resource utilization within the constraints of the target device. Each of the three processing stages relies heavily on multiply-accumulate operations, and the FPGA provides a finite number of hard DSP blocks. With all three algorithms competing for the same resource pool, careful consideration was required to determine how DSP blocks should be allocated across the system and to what degree each algorithm should be parallelised. Increasing parallelism improves throughput and reduces latency but consumes more DSP blocks, while a more sequential implementation conserves resources at the cost of processing bandwidth. Resolving these tradeoffs required the team to reason about the relative latency requirements of each stage and the degree to which operations could be time-multiplexed onto shared hardware. This process reinforced the importance of resource budgeting early in the design cycle, before microarchitectural decisions have been made that implicitly commit to a particular level of parallelism.

6. Contributions of Individuals

The following lists outline the specific hardware pipelines and software modules each team member contributed to over the course of the project.

Avery: audio control and codec interfacing, autocorrelation, PSOLA, mode/note selection logic, system integration
Kalyan: preprocessing and filters, autocorrelation, PSOLA, harmonizer, system integration
Vishrut: midi keyboard integration, autocorrelation, all web development and UX
Jonathan: all python software models, filter design, vocoding, system testing